



AUSSIE AI

Integer Overflow and Underflow

Book Excerpt from "Generative AI in C++"
by David Spuler, Ph.D.

Integer Overflow and Underflow

Integer arithmetic overflow and underflow have traditionally been ignored in C++ programs, mostly by assuming that operations won't exceed the range of 32-bit integers. Most platforms don't fail on integer overflow, and quietly continue, without even triggering a signal like **SIGFPE** (floating-point error).

The absence of runtime warnings can potentially leave insidious bugs in your code, and is also an undefended attack vector for security. Also, perhaps ignoring overflow isn't the best strategy if you're using integer operations for your AI model, such as with integer quantization. On the other hand, there's this weird stochastic feature of AI models, which is that they often get *better* when errors occur occasionally, because some randomness can be helpful. Even so, it's better to know what's going on.

Integers have a fixed range of numbers that they can represent. For example, a signed 16-bit integer represents the relatively small range of **-32,768** to **+32,767**, and an unsigned 16-bit number can be from **0** to **65,535**. A 32-bit signed integer has a much bigger range from about negative 2 billion (–



from zero up to about 4 billion (+4,294,967,295). Feel free to memorize those numbers, as you'll be needing them at least once a decade. The ranges for 64-bit integers are massive numbers around 2^{64} , which is approximately decimal 10^{19} .

If integer arithmetic on a data type falls outside the range supported by that integer type, then an overflow or underflow occurs. There are symbolic constants for the minimum and maximum numbers for many types declared in the `<limits.h>` standard header file.

- `int` — `INT_MAX` and `INT_MIN`
- `unsigned int` — `UINT_MAX` and `UINT_MIN`

The effect of integer overflow or underflow is platform-specific, but on most platforms, it is usually: *nothing!* It's a silent insidious bug in many cases. For a signed integer, overflow quietly wraps around from positive to negative, and underflow does the reverse.

Here's an example of overflow of an `int` type:

```
int x = INT_MAX;
assert(x >= 0);
++x; // Overflow!
assert(x < 0);
```

And this is underflow of `int`:

```
int x = INT_MIN;
assert(x < 0);
--x; // Underflow!
assert(x > 0);
```

Floating-point types can represent much larger magnitude numbers than integers. Hence, another way for an integer to overflow is in a conversion from floating-point numbers.



For an unsigned integer, the results are a little different, since negatives are not possible. Instead, overflow wraps around from a large number to zero, and underflow (going below zero) wraps around to the largest unsigned number.

Preventing Integer Arithmetic Overflow. There's not really a good way to detect arithmetic overflow or underflow before it happens. Post-testing is easier.

For example, GCC and Clang have some intrinsics, such as “`__builtin_add_overflow`” for addition, which use post-testing of the x86 CPU overflow or carry flags for detecting integer overflow, and return a Boolean flag which you can use. The GCC documentation say it uses “conditional jump on overflow after addition” and “conditional jump on carry” for unsigned overflow. Here's an example:

```
if (__builtin_add_overflow(x, y, &z)) {  
    // Overflow!  
}
```

The mainstream prevention strategy is simply to choose a big integer type (at least 32-bit) and then hope that no outliers occur in your input data. Most programmers let the overflow occur and then check. Or rather, just between you and me, most programmers simply don't even check at all!

Technically, integer overflow is “undefined behavior” on C++, and it's certainly non-portable, so you really should check. But most platforms handle it the same way, by quietly wrapping the integers around in two's complement form.

Increment overflow. For incrementing integers, you can do a pre-test like:



```
}  
else {  
    x++; // Safe increment  
}
```

Addition overflow. And here's a version to pre-test addition of two positive integers for overflow:

```
if (x > INT_MAX - y ) { // x + y > INT_MAX  
    // Overflow!  
}  
else {  
    x += y; // Add safely  
}
```

Multiplication overflow. The test for multiplication overflow is even worse because it uses division:

```
if (x > INT_MAX / y ) { // x * y > INT_MAX  
    // Overflow!  
}  
else {  
    x *= y; // Multiply safely  
}
```

Head in the sand approach. Unfortunately, pre-testing for overflow is massively inefficient, as shown above. Do you really want to do this for every addition or increment? Even post-testing for overflow isn't much better. Overall, there's good reason why most C++ programmers just skip it, and hope for the best.

Overflow management. The alternative to ignoring the problem is to consider various different risk mitigation strategies for integer overflow:

- Larger data types (e.g. `long`) for a larger range.
- Use floating-point types instead.



- Use `size_t` for the `unsigned` variable type (it's standardized).
- Enable compiler runtime checks (when debugging/testing)
- Range checking input numbers (e.g. model weights).
- Post-testing the sign of arithmetic results.
- GCC and Clang intrinsic functions with overflow testing.
- The `<stdckdint.h>` header file in C23 (that's the C standard, not C++23).
- Safe integer class wrappers.

Runtime overflow detection. Some C++ compilers provide limited support for runtime error checking of arithmetic. The x86 CPU has builtin overflow detection, with a quietly-set overflow flag and a carry flag, which some C++ compiler-writers have made use of.

GCC has an “`-ftrapv`” option which elevates overflow errors (presumably by using post-checking). GCC has defined a number of C++ intrinsic functions which you can use to perform overflow-safe integer arithmetic, such as:

- `__builtin_add_overflow` — addition
- `__builtin_mul_overflow` — multiplication

Microsoft Visual Studio C++ provides the “`/RTC`” option, which stands for “Run-Time Checks”, or there's “Basic Runtime Checks” in the MSVS IDE Project Settings. However, these MSVS features don't check much for arithmetic overflow, with a focus on stack frame checking and uninitialized variables. The closest is “`/RTCc`” to detect data type truncations at runtime.

There's also a runtime debugging tool that focuses on integer overflow and other oddities. It's named “Undefined Behavior



Safe integer classes. Currently there's no standard safe integer types in C++, but adding them was unsuccessfully proposed in 2016. If you like a busy CPU, and what AI programmer doesn't, you can replace all `int` variables with "safe integer" class objects, with many examples of such classes available on the Internet. They're probably not as bad as I've implied, since C++ inlining should make the critical path quite short.

- Next:
- Up: [Table of Contents](#)
- Buy: [Generative AI in C++: Coding Transformers and LLMs](#)



The new [AI programming book](#) by Aussie AI co-founders:

- AI coding in C++
- Transformer engine speedups
- LLM models
- Phone and desktop AI
- Code examples
- Research citations

Get your copy from Amazon: [Generative AI in C++](#)



Turn Your AI Upside-Down

QUICK LINKS

[About Us](#)

[Login](#)

[Sign Up](#)

PRODUCT

[Plans and Pricing](#)

[Try Aussie AI For Free](#)

[Research](#)

NEW TO WRITING?

[How to Write Your](#)

[Novel](#)

[Choices in Novel](#)

[Writing](#)

[Terms Writers Should](#)

[Know](#)

WRITING STYLES

[Immersive Writing](#)

[Newer Writing Styles](#)

[Polishing Your First](#)

[Draft](#)

Copyright © 2026 Aussie AI. All rights reserved.

[Terms of Use](#) [Privacy Policy](#) [Notices](#)